Octobre 2026

Rapport de TP

Ingerer les donnees simulees & Rechercher (SOC L1)

Pape Aldiouma SARR
L3 Cybersecurite - Module : Ethical Hacking / SOC

Rapport de travaux pratiques : TP 2, Ingerer les donnees simulees et rechercher. Parcours SaaS : template ECS, upload de datasets, Data View, KQL, cle API.



Republique du Senegal, Un Peuple, Un But, Une Foi
Ministere de l'Enseignement Superieur, de la Recherche et de l'Innovation
Direction Generale de l'Enseignement Superieur Prive
Institut Superieur de Management
Annee academique 2026-2027

Table des matieres

Partie I : Presentation du travail....3
Partie II : Appliquer le template ECS....4
Partie III : Uploader les datasets....6
Partie IV : Verifier l'ingestion (Dev Tools)....10
Partie V : Creer la Data View....13
Partie VI : Recherche KQL....14
Partie VII : Cle API restreinte....18
Partie VIII : Defis (autonomie)....22
Conclusion....27

Partie I : Presentation du travail

Contexte

A l'issue du TP1, la plateforme SOC-LAB-GLACIER (Elastic Cloud Hosted) est operationnelle mais vide : aucun index de donnees n'existe encore. Ce TP2 enchaine directement dessus avec pour objectif d'injecter des evenements ECS simules (trafic normal et scenarios d'attaque, livres sous forme de fichiers NDJSON) puis de commencer a les interroger. Les memes datasets que le parcours auto-heberge sont utilises, l'objectif etant de comparer les deux methodes d'ingestion plutot que le contenu des donnees lui-meme.

Objectifs pedagogiques

- Appliquer un template d'index ECS pour garantir des mappings corrects.
- Ingerer des datasets NDJSON via Kibana, sans outil local.
- Verifier l'ingestion dans Dev Tools (index, volumetrie, mappings).
- Creer une Data View et l'utiliser dans Discover.
- Ecrire des requetes KQL et borner le temps.
- Creer une cle API restreinte (moindre privilege).
- Identifier des IOC a partir des evenements.

Datasets fournis

Fichier	Evenements	Contenu
sample-normal.ndjson	300	Trafic legitime uniquement
sample-mixed.ndjson	1000	85% normal / 10% suspect / 5% malveillant
scenario-brute-force.ndjson	60	Echecs SSH repetes
scenario-port-scan.ndjson	80	Scan multi-ports (pare-feu)
scenario-web-attack.ndjson	60	SQLi, path traversal, User-Agent offensif
tp5-incident.ndjson	~1195	Kill chain complete, reservee au TP5

Conformement a la consigne, les cinq premiers datasets (hors tp5-incident.ndjson, reserve au TP5) ont ete uploades, pour un total attendu de 1500 evenements (300 + 1000 + 60 + 80 + 60).

Point de controle 1

Chaque dataset a ete associe a son scenario avant l'ingestion : trafic normal (sample-normal, sample-mixed), brute force SSH, scan de ports, et attaque web. Le fichier tp5-incident.ndjson a ete volontairement laisse de cote pour le TP5.

Partie II : Appliquer le template ECS

Pourquoi un template avant l'ingestion

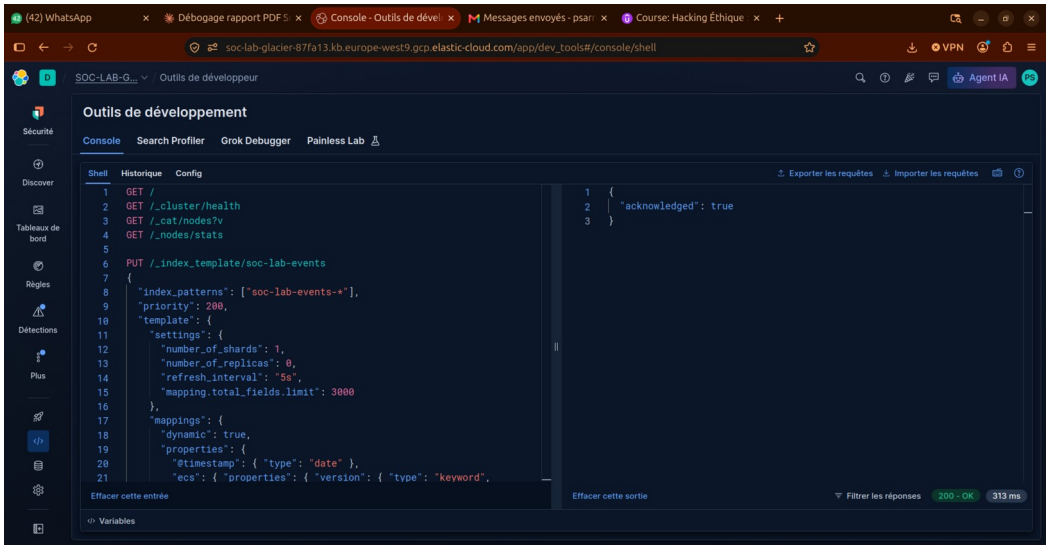
Un template d'index force les bons types (keyword, ip, date) au moment de la creation d'un index correspondant a son pattern. Sans lui, un upload avec inference automatique de mapping typerait la plupart des champs en text, ce qui casse les agregations exactes, les recherches par plage sur les adresses IP (CIDR) et, plus generalement, fiabilise moins les regles de detection construites plus tard (TP4).

Application du template

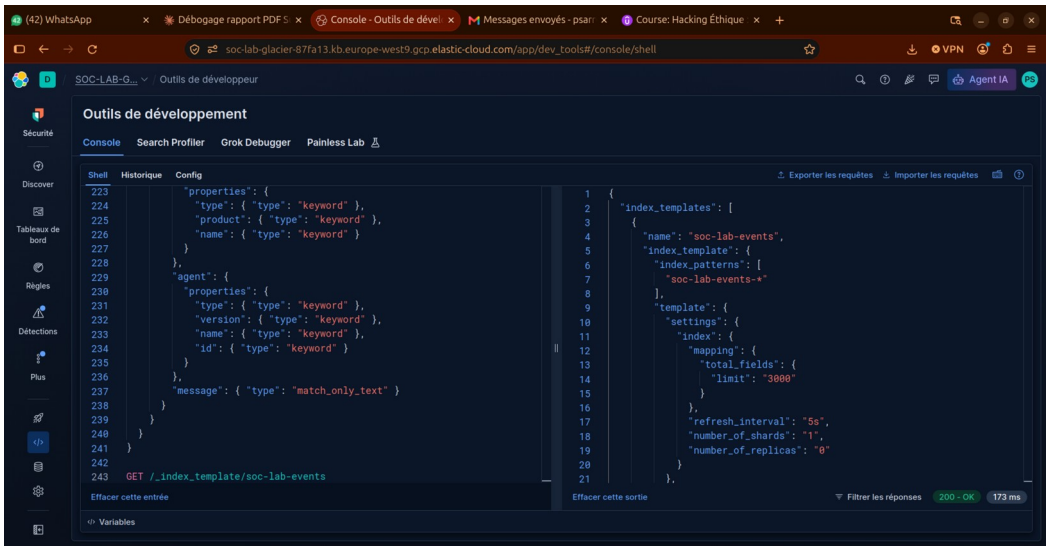
Le contenu complet du fichier index-template.json (236 lignes, mappings ECS couvrant @timestamp, event.*, host.*, user.*, source.*/destination.*, network.*, process.*, file.*, url.*, http.*, rule.*, dns.*, registry.*, winlog.*, log.*, service.*, observer.*, agent.* et message en match_only_text) a ete colle dans Dev Tools et execute avec la commande suivante :

```
PUT /_index_template/soc-lab-events
{
  "index_patterns": ["soc-lab-events-*"],
  "priority": 200,
  "template": { "settings": { ... }, "mappings": { ... } }
}
```

La reponse a confirme l'application du template : {"acknowledged": true}, code 200 OK. Une verification immediate a ete effectuee avec GET /_index_template/soc-lab-events, qui a renvoye 200 OK avec l'integralite du contenu colle, confirmant que le template est bien enregistre cote cluster avant toute ingestion.



Dev Tools : PUT /_index_template/soc-lab-events, reponse {"acknowledged": true}, 200 OK en 313 ms.



Dev Tools : GET /_index_template/soc-lab-events, le template enregistre est renvoye en integralite (index_patterns, settings, mappings), 200 OK en 173 ms.

Signification de index_patterns (Q4.1)

La valeur ["soc-lab-events-*"] indique qu'Elasticsearch appliquera automatiquement ce template a tout index cree dont le nom commence par soc-lab-events-, quel que soit le suffixe utilise ensuite (une date comme 2026.10.18, un nom libre, etc.). C'est ce mecanisme de correspondance par motif qui garantit que chaque nouvel index de donnees du lab, meme cree via l'assistant d'upload sans intervention manuelle, herite automatiquement des bons mappings.

Pourquoi source.ip en type ip plutot que text (Q4.2)

Le type ip permet a Elasticsearch de comprendre la semantique d'une adresse IPv4/IPv6 : il autorise les requetes par plage et par notation CIDR (par exemple source.ip: "10.10.0.0/16"), des agregations exactes (Top values, cardinalite) et un stockage compact. Un mapping text, au contraire, decoupe la chaine en tokens separes par les points (tokenisation standard), ce qui detruit completement la structure de l'adresse : une recherche sur une plage ou un sous-reseau devient impossible, et les valeurs distinctes ne sont plus comptees correctement.

Consequence de l'absence du template a l'upload (Q4.3)

Sans template prealable, le premier document uploade declencherait l'inference de mapping dynamique d'Elasticsearch : les champs numeriques et les adresses IP seraient tres probablement types en text ou en champs multiples non adaptes (selon les heuristiques d'inference de l'assistant d'upload), et ce mapping, une fois fixe par le premier document indexe, s'appliquerait a tous les documents suivants du meme index. Corriger un mapping errone apres coup necessite de reindexer entierement les donnees dans un nouvel index, une operation couteuse evitee ici en appliquant le template avant tout upload.

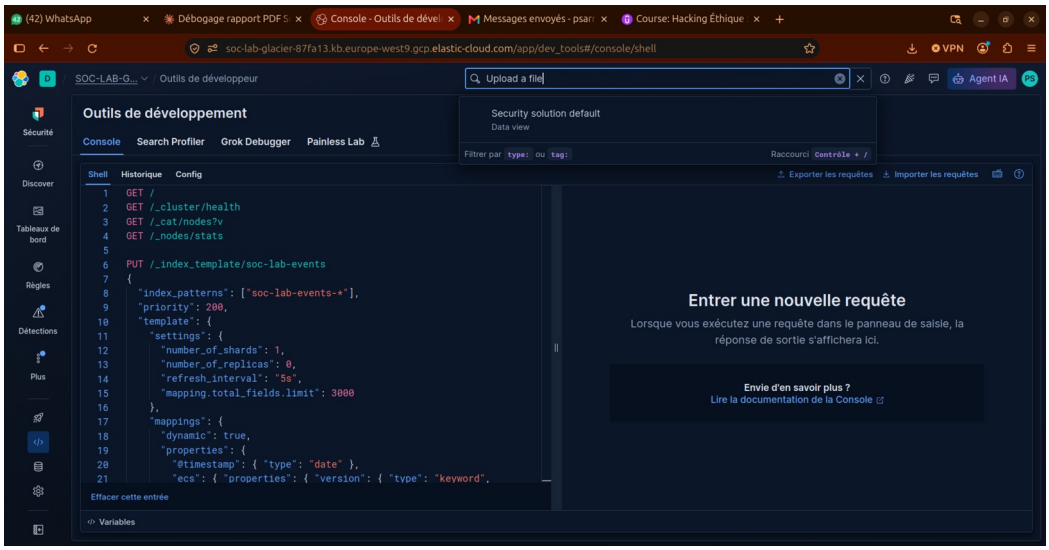
Point de controle 2

Le template soc-lab-events existe et repond 200 OK a la requete GET /_index_template/soc-lab-events, avec l'integralite des mappings ECS attendus.

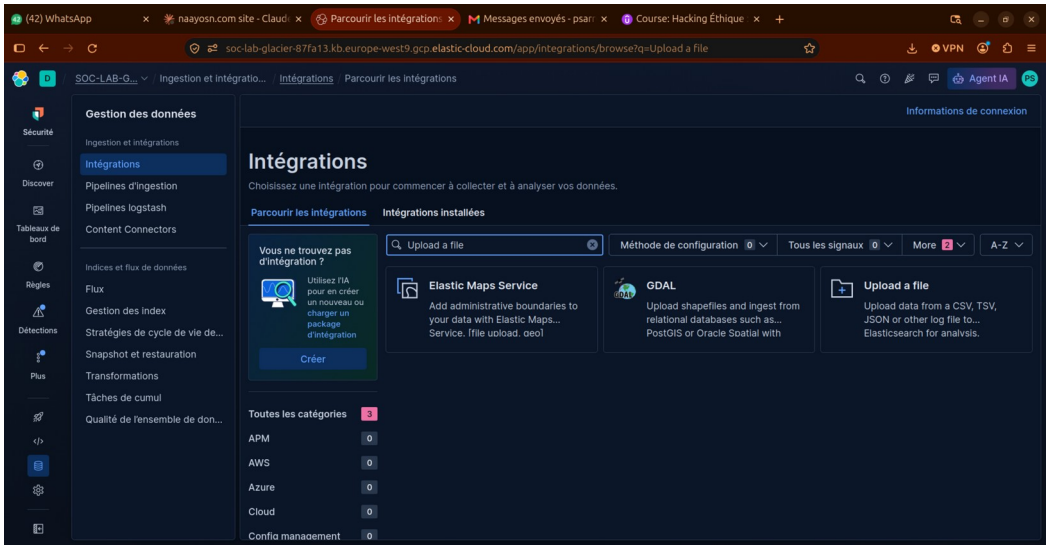
Partie III : Uploader les datasets

Acces a l'outil d'upload

La fonctionnalite Upload a file (Data Visualizer) n'a pas ete trouvee via la barre de recherche globale de Kibana : taper Upload a file n'a remonte qu'un resultat sans rapport (une Data View existante). L'acces a ete obtenu en naviguant directement vers /app/integrations/browse, ou la carte Upload a file est effectivement presente et accessible.



Barre de recherche globale de Kibana : la saisie "Upload a file" ne remonte qu'une Data View existante, sans rapport avec l'outil recherche.

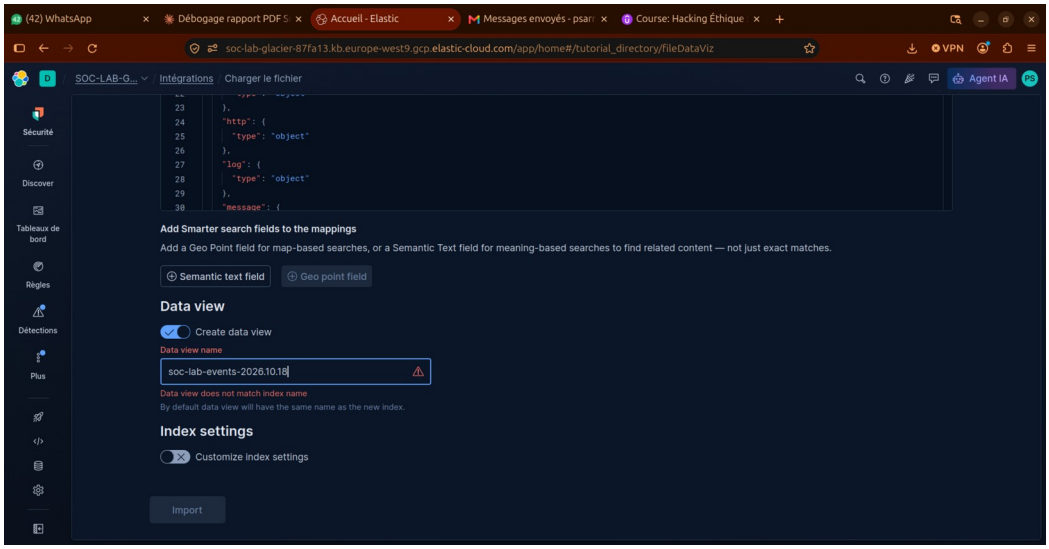


Page Parcourir les integrations (/app/integrations/browse) : la carte "Upload a file" (donnees CSV, TSV, JSON...) y est bien presente et accessible.

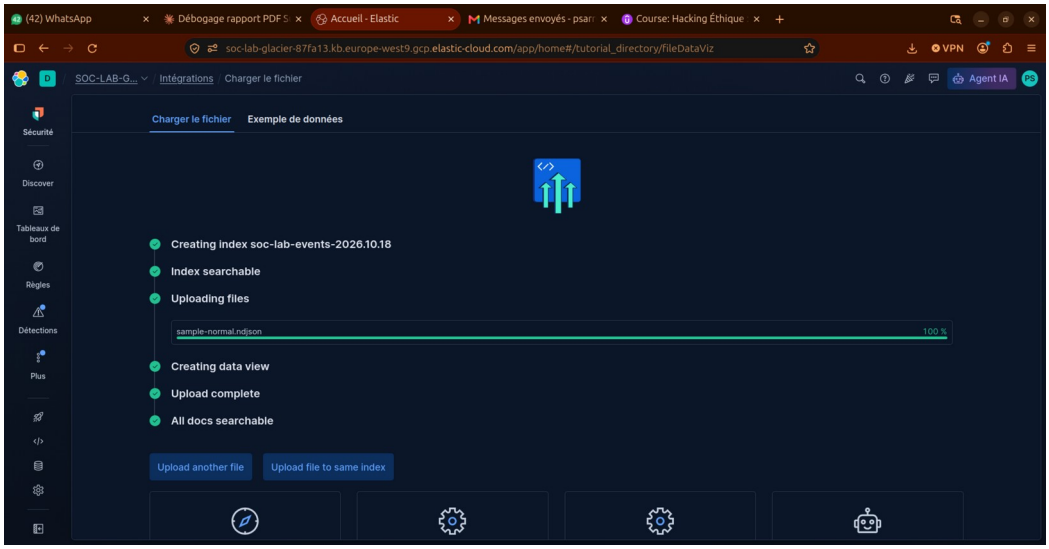
Premier upload et correction d'une erreur de saisie

Lors du premier import (sample-normal.ndjson), apres previsualisation du fichier, l'onglet Advanced a ete utilise pour nommer l'index. Une confusion s'est produite a cette etape : le champ Data view name a ete rempli avec soc-lab-events-2026.10.18, mais le champ Index name, situe plus haut dans le formulaire et distinct du precedent, avait ete laisse vide. Kibana a signale l'erreur par un message rouge Data view does not match index name. Apres avoir identifie que ces deux champs sont independants (le nom d'index cree physiquement l'index Elasticsearch et doit respecter le pattern soc-

lab-events-* du template ; le nom de Data view est un simple libelle d'affichage, generalement synchronise automatiquement), le champ Index name a ete correctement renseigne et l'import a abouti : Creating index, Index searchable, Uploading files 100%, Creating data view, Upload complete.



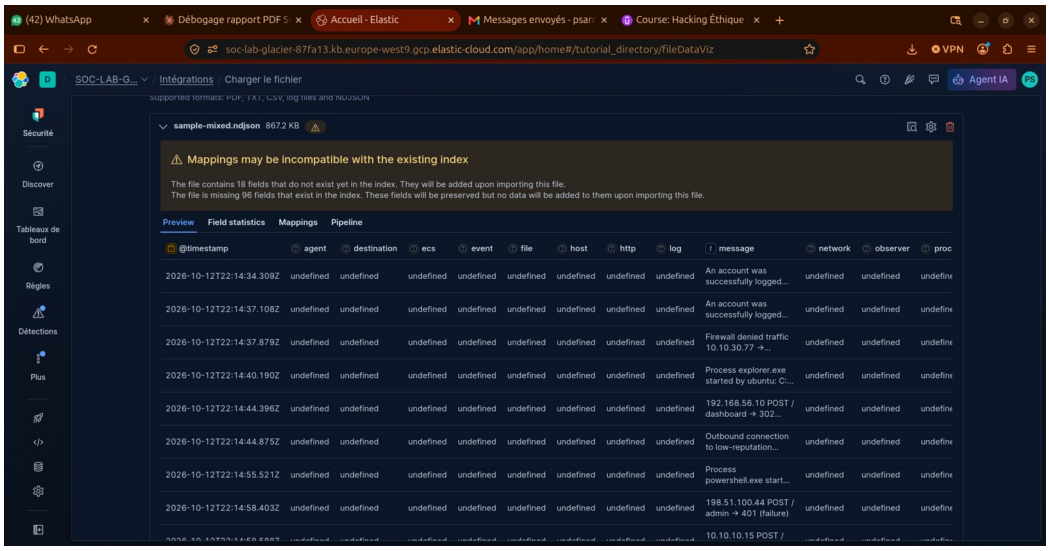
Message d'erreur rouge "Data view does not match index name" : le champ Data view name a ete rempli avant le champ Index name, laisse vide.



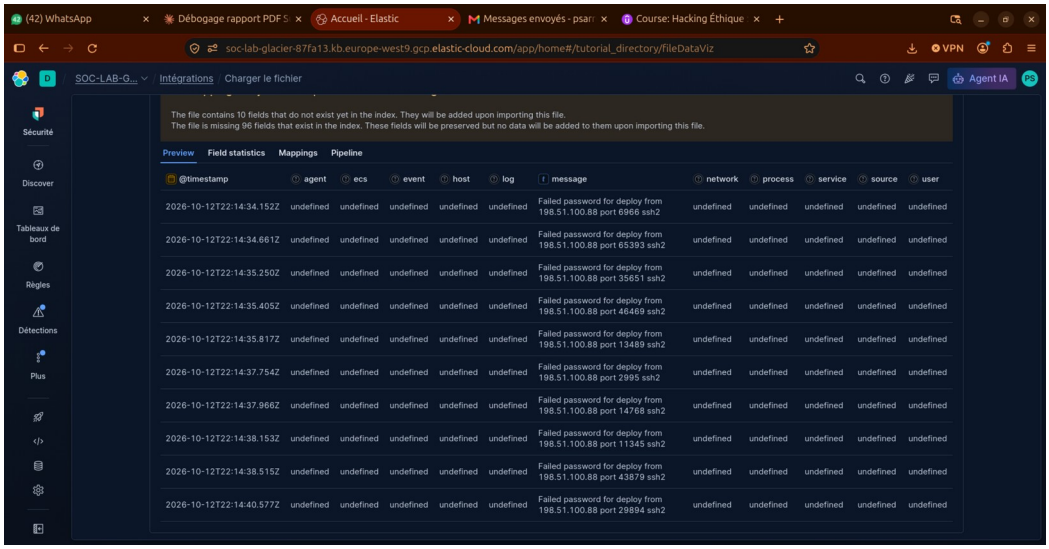
Apres correction du champ Index name : Creating index soc-lab-events-2026.10.18, All docs searchable, import reussi.

Imports suivants dans le meme index

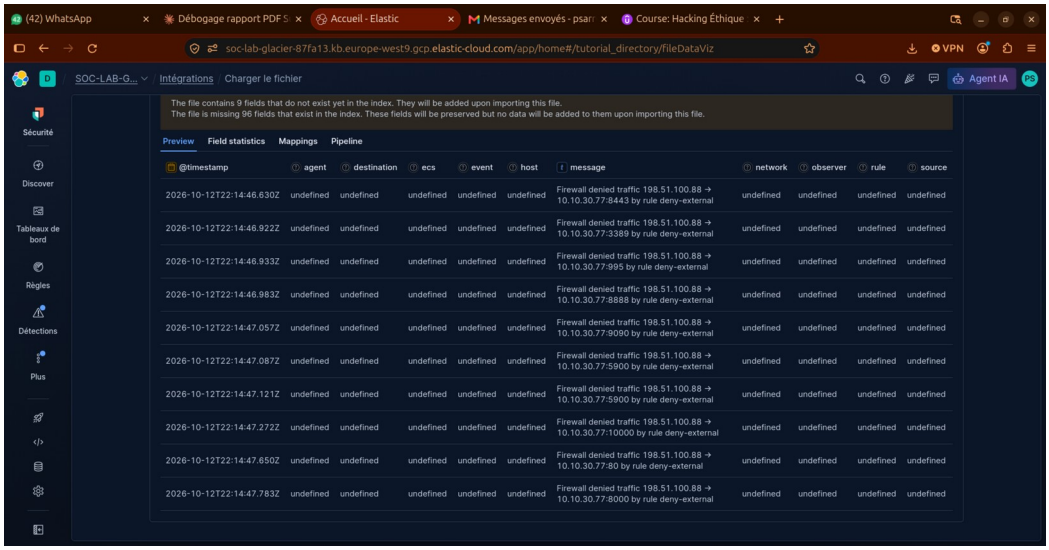
Les quatre autres datasets (sample-mixed, scenario-brute-force, scenario-port-scan, scenario-web-attack) ont ete uploades a la suite, dans le meme index soc-lab-events-2026.10.18, via le bouton Upload file to same index. Chaque import a affiche un bandeau d'avertissement Mappings may be incompatible with the existing index (par exemple 18 nouveaux champs / 96 champs manquants pour sample-mixed), ce qui n'est pas une erreur : chaque dataset ne remplit naturellement qu'un sous-ensemble des quelque 100 champs ECS predeclares par le template (dynamic: true permettant par ailleurs l'ajout automatique de tout champ reellement nouveau).



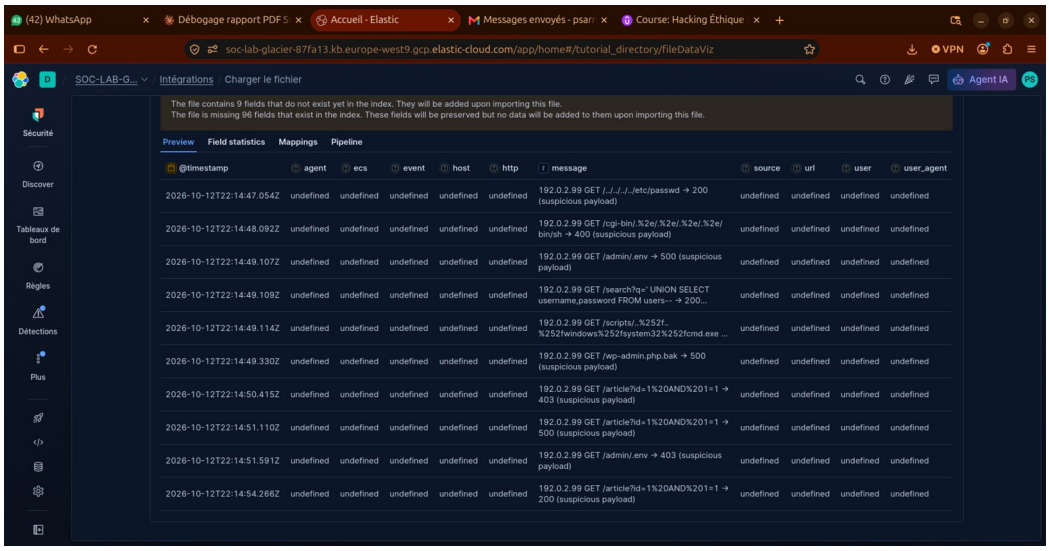
Previsualisation de sample-mixed.ndjson : avertissement "Mappings may be incompatible" (18 champs nouveaux / 96 manquants), non bloquant.



Previsualisation de scenario-brute-force.ndjson : echecs SSH repetes sur le compte deploy, meme avertissement de mapping non bloquant.



Previsualisation de scenario-port-scan.ndjson : trafic bloque par la regle deny-external sur FW-EDGE01, meme avertissement de mapping.



Previsualisation de scenario-web-attack.ndjson : payloads d'injection (/etc/passwd, UNION SELECT..., traversee de repertoire), meme avertissement de mapping.

Pourquoi l'index doit commencer par soc-lab-events- (Q5.1)

C'est la condition necessaire et suffisante pour que le template applique en Partie II s'active automatiquement au moment de la creation de l'index : le pattern index_patterns du template n'accepte que les noms commencant par ce prefixe. Un index nomme differemment serait cree avec le mapping dynamique par defaut d'Elasticsearch, sans beneficier des types explicites (ip, keyword) configures.

Inference de mapping contre template (Q5.2)

L'inference de mapping est une estimation automatique, realisee document par document au moment de l'upload, qui devine le type de chaque champ a partir des premieres valeurs rencontrees (souvent text par defaut pour les chaines). Le template, a l'inverse, est une declaration explicite et anterieure a toute ingestion : il garantit un typage identique et predictible quel que soit l'ordre ou le contenu des fichiers importes ensuite, ce qui est indispensable pour que les regles de detection (TP4) et les agregations restent fiables sur la duree.

Upload par fichier et flux de production (Q5.3)

Non : l'upload par fichier est explicitement concu par Elastic pour la decouverte initiale et l'exploration ponctuelle de donnees (demonstration, prototypage), pas pour un flux continu. Il impose une action manuelle a chaque fichier, une limite de taille de 100 Mo par fichier, et ne gere ni planification ni retry automatique. Un flux de production reel s'appuierait sur un pipeline automatise (agent Elastic, Logstash, ou l'API _bulk utilisee plus loin dans ce TP, Partie VIII, Defi D).

Point de controle 3

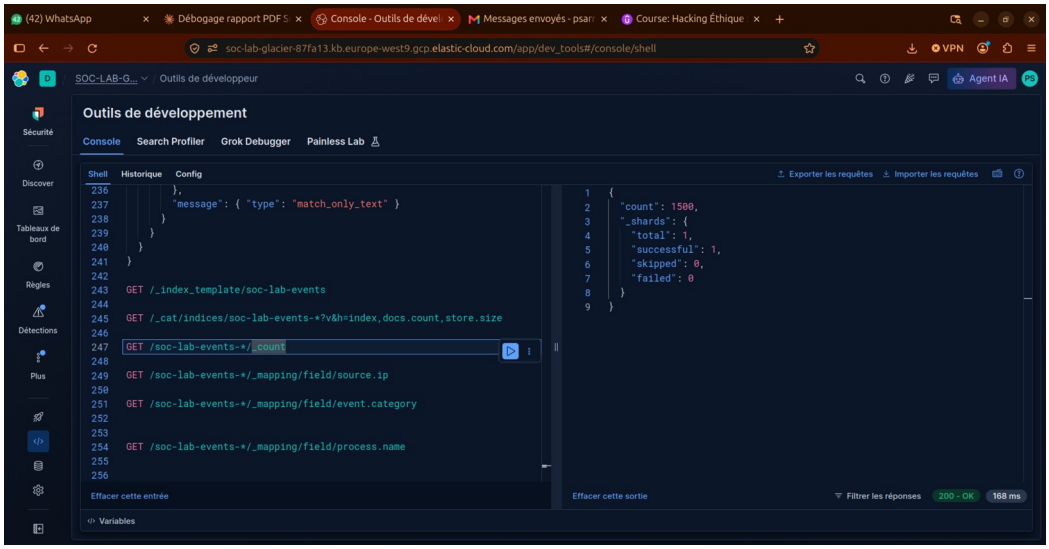
Les cinq datasets (hors tp5-incident.ndjson) sont visibles dans l'apercu de l'index soc-lab-events-2026.10.18, chacun ayant ete importe avec succes malgre les avertissements de mapping, non bloquants.

Partie IV : Verifier l'ingestion (Dev Tools)

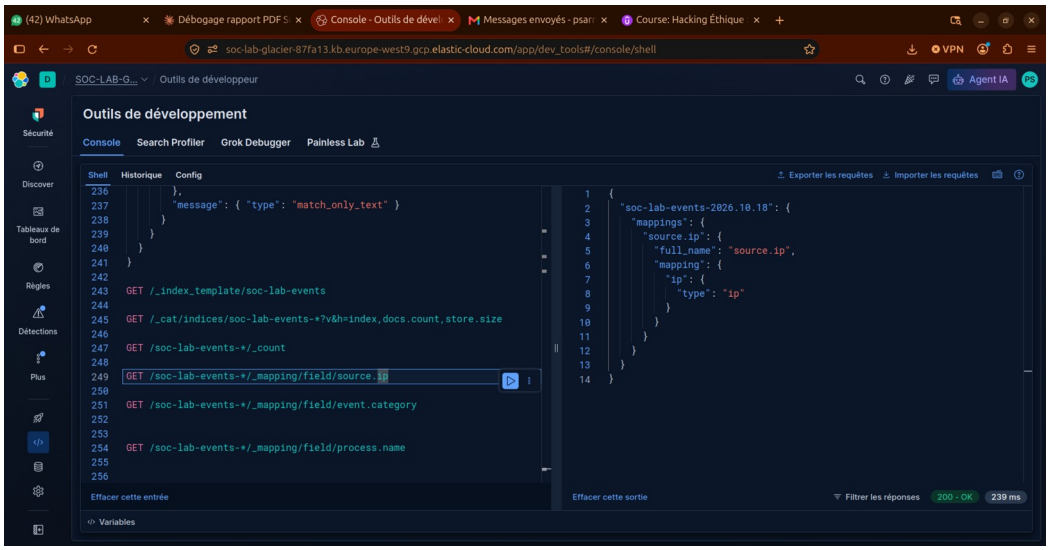
Cinq requetes de verification ont ete executees dans Dev Tools apres les cinq uploads :

```
GET /_cat/indices/soc-lab-events-*?v&h=index,docs.count,store.size
GET /soc-lab-events-*/_count
GET /soc-lab-events-*/_mapping/field/source.ip
GET /soc-lab-events-*/_mapping/field/event.category
GET /soc-lab-events-*/_mapping/field/process.name
```

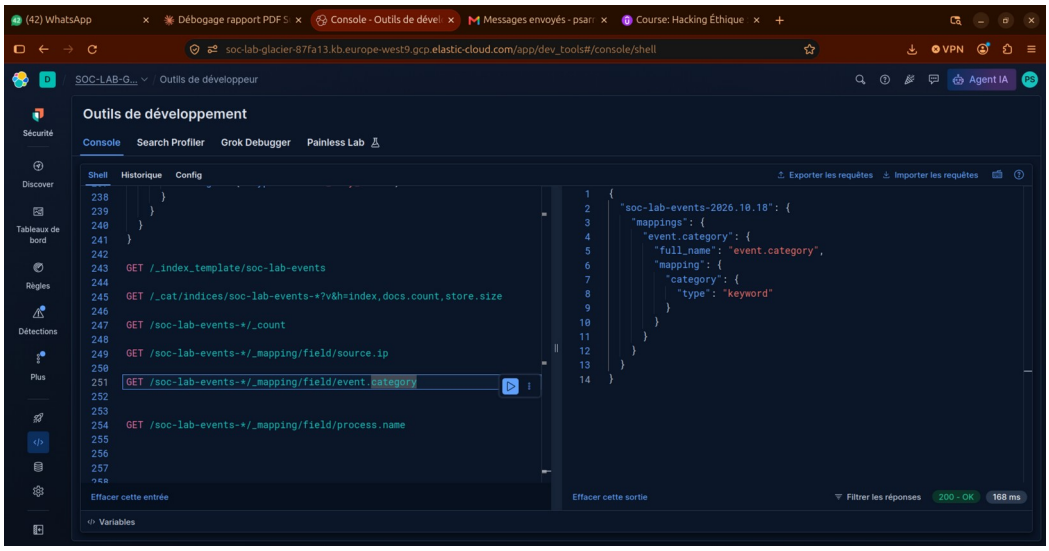
Resultats : l'index unique soc-lab-events-2026.10.18 contient 1500 documents pour une taille de stockage de 689,3 ko, confirmant l'addition exacte des cinq datasets (300 + 1000 + 60 + 80 + 60). Le comptage via /_count confirme egalement 1500, avec tous les shards en succes.



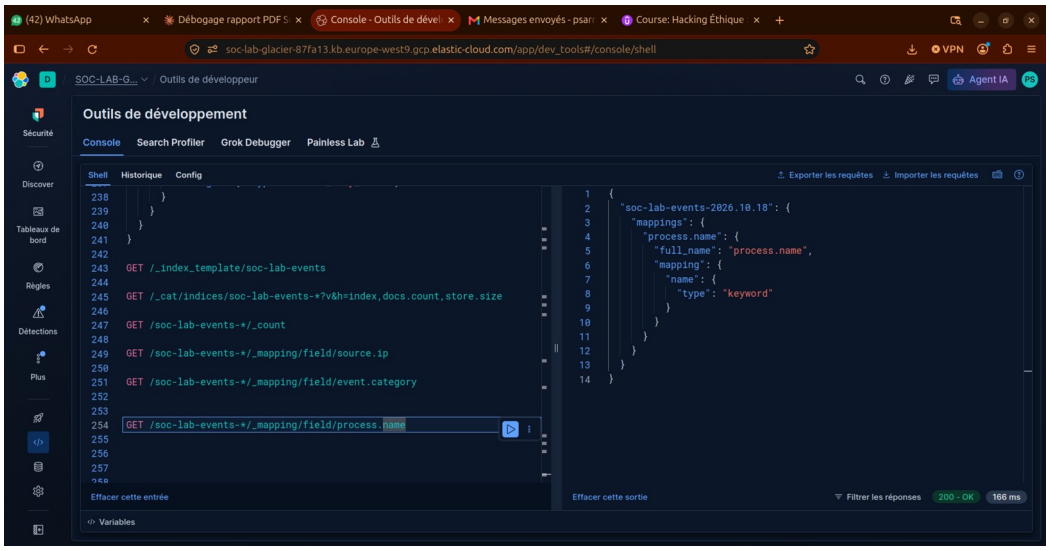
Dev Tools : GET /_cat/indices/soc-lab-events-?v&h=index,docs.count,store.size -> soc-lab-events-2026.10.18, 1500 docs, 689.3kb.*



Dev Tools : GET /soc-lab-events-/_count -> {"count": 1500}, tous les shards en succes.*



Dev Tools : GET .../_mapping/field/event.category -> type keyword confirme.



Dev Tools : GET .../_mapping/field/process.name -> type keyword confirme.

Volume total et nombre d'index (Q6.1)

1500 documents au total, repartis dans un seul index (soc-lab-events-2026.10.18), puisque les cinq uploads ont cible le meme index plutot que cinq index distincts.

Types des trois champs verifies (Q6.2)

Champ	Type confirme
source.ip	ip
event.category	keyword
process.name	keyword

Ces trois types correspondent exactement a ceux declares dans le template de la Partie II, preuve que le template a bien ete applique avant le premier import.

Un meme index pour plusieurs scenarios (Q6.3)

Oui, un meme index peut parfaitement contenir des documents issus de scenarios differents : c'est exactement ce qui a ete demontre ici, en uploadant successivement cinq datasets distincts (trafic normal, brute force, scan de ports, attaque web, trafic mixte) dans le meme index soc-lab-events-2026.10.18. Elasticsearch ne distingue pas

les documents par leur provenance mais uniquement par leur contenu ; c'est la raison pour laquelle aucun champ du schema ECS n'indique explicitement le scenario d'origine, l'analyste devant le deduire de l'analyse (point rappele dans le README des datasets en vue du TP5).

Point de controle 4

Confirme : source.ip est de type ip, event.category et process.name sont de type keyword, sur un total de 1500 documents dans un index unique.

Partie V : Creer la Data View

La Data View a ete creee depuis Stack Management > Data Views > Create data view, avec les parametres imposes par l'enonce : Name = SOC-LAB Events (simules), Index pattern = soc-lab-events-*, Time field = @timestamp, et surtout un Custom data view ID fixe a soc-lab-events (obtenu en ouvrant le lien Set a Custom data view ID, cache par defaut sous Afficher les parametres avances).

Un bug d'URL rencontre et corrige

En tentant de naviguer directement vers la page des Data Views par une URL tapee a la main, une double barre oblique s'est glissee dans l'adresse (…elastic-cloud.com//app/management/kibana/dataViews). Kibana a repondu par une page d'erreur JSON brute ({"statusCode":404,"error":"Not Found","message":"Not Found"}) plutot que par l'interface attendue, le routage serveur ne reconnaissant pas ce chemin malforme. La correction a consiste a retirer la barre oblique en trop ; la navigation a ensuite ete privilegiee via le menu lateral Stack Management plutot que par saisie manuelle d'URL, pour eviter de reproduire ce type d'erreur.

Data View contre index (Q7.1)

L'index est la structure de stockage physique d'Elasticsearch : c'est la ou les documents sont effectivement ecrits et indexes. La Data View est un objet propre a Kibana, une couche d'abstraction qui reference un ou plusieurs index (via un pattern) pour permettre leur exploitation dans l'interface : Discover, visualisations Lens, dashboards. Sans Data View, aucune de ces applications Kibana ne peut interroger les index, meme si les donnees y sont presentes et accessibles via l'API brute (Dev Tools).

Pourquoi un ID personnalise plutot qu'aleatoire (Q7.2)

Les dashboards construits au TP4 referencent une Data View par son identifiant technique (ID), pas par son nom affiche. Si la Data View avait ete creee avec l'ID aleatoire genere par defaut, tout dashboard important plus tard et reference a l'ID attendu (soc-lab-events) se retrouverait vide, faute de correspondance. Fixer l'ID des maintenant evite ce piege silencieux, qui ne se manifeste qu'au moment de l'import des dashboards, bien plus tard dans le fil rouge.

Pourquoi @timestamp est le champ de temps attendu (Q7.3)

@timestamp est le champ temporel standard du schema ECS, present de maniere identique dans tous les documents, quel que soit leur scenario ou leur module d'origine. C'est ce champ que Kibana utilise pour construire l'histogramme temporel de Discover, pour appliquer une plage de temps (filtre global) et pour trier chronologiquement les resultats. Utiliser un autre champ de date, propre a un seul type d'evenement, romprait cette coherence des qu'un dataset different serait affiche dans la meme vue.

Plage de temps et donnees figees

Une fois la Data View enregistree, Discover est apparu vide a la premiere ouverture : la plage de temps par defaut (Last 15 minutes) ne couvre evidemment pas des evenements simules et generes a une date fixe (12 octobre 2026). Le passage a Last 7 days a immediatement fait apparaitre les 1500 documents, avec un pic net dans l'histogramme concentre sur une fenetre de quelques heures le 12 octobre.

Point de controle 5

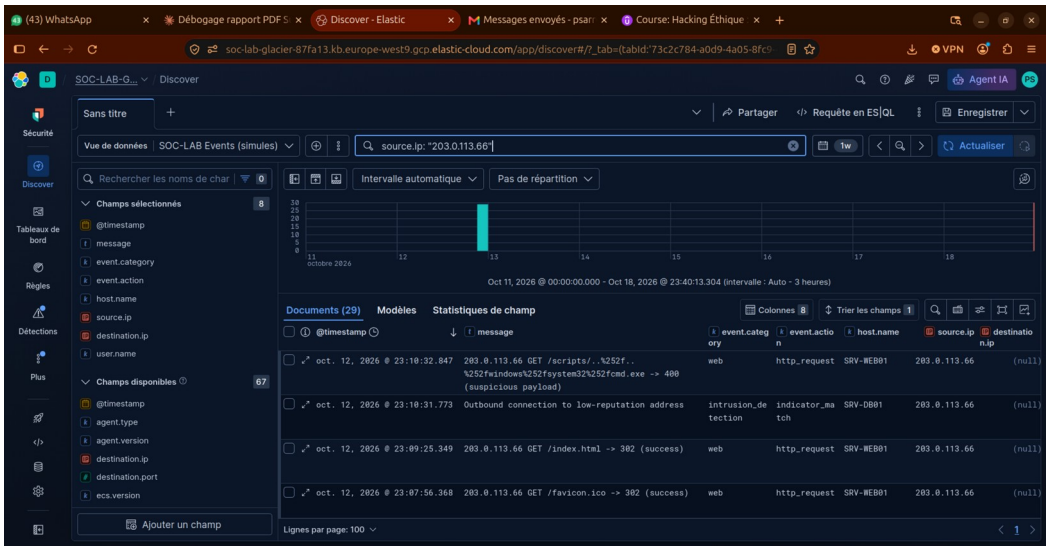
La Data View existe avec l'ID personnalise soc-lab-events, le pattern soc-lab-events-* et le champ temporel @timestamp ; elle affiche les 1500 documents une fois la plage de temps elargie a Last 7 days.

Partie VI : Recherche KQL

8.1 - Premiers filtres

Trois requetes KQL successives ont ete executees dans Discover (Data View SOC-LAB Events, plage Last 7 days) :

```
host.name: "SRV-WEB01"                                          -> 488 documents
event.category: "authentication" and event.outcome: "failure"   -> 165 documents
event.category: "web" and http.response.status_code: 403        -> 18 documents
```



Discover, requete event.category: "authentication" and event.outcome: "failure", 165 documents, colonnes event.category/event.action/host.name/source.ip mises en surbrillance.

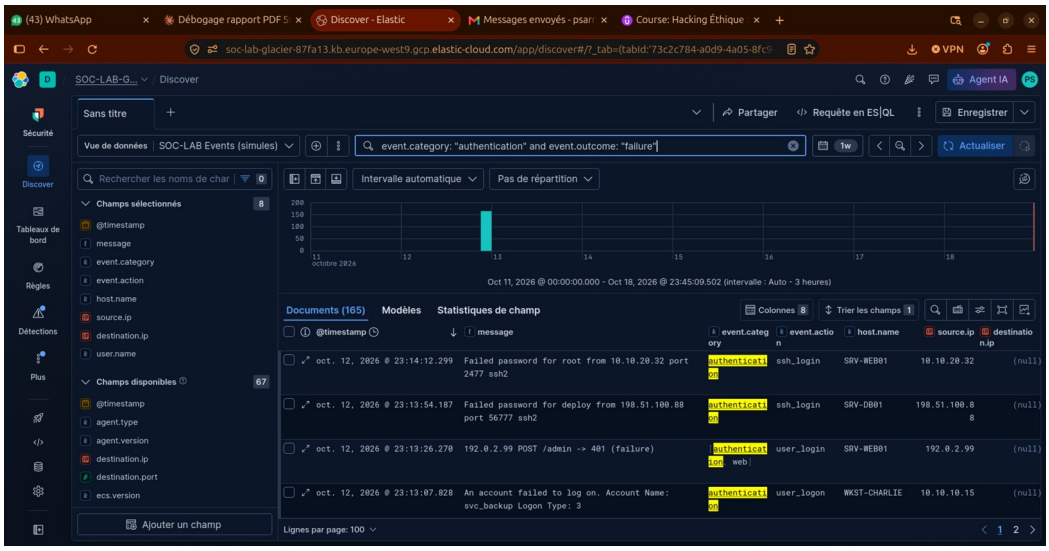
La troisieme requete (echecs web 403) a revele des signatures nettes d'attaque applicative contre SRV-WEB01 : tentatives d'injection SQL, de traversee de repertoire et de sondage de fichiers sensibles, en provenance de trois adresses IP distinctes (198.51.100.23, 203.0.113.66, 192.0.2.99).

Volume d'echecs d'authentification sur 7 jours (Q8.1)

165 documents correspondent au filtre event.category: "authentication" and event.outcome: "failure" sur la fenetre de 7 jours.

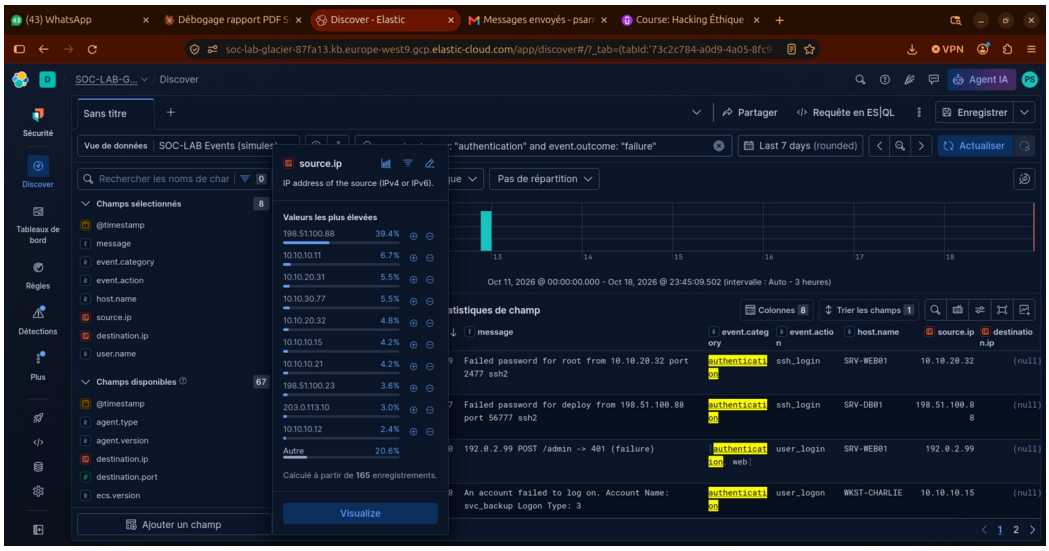
8.2 - Suivre une IP source

Premiere tentative, erronee : en reprenant l'adresse 203.0.113.66 deja observee dans les resultats de la requete web/403 precedente (au lieu de la lire specifiquement dans le Top values du filtre d'authentification), la requete source.ip: "203.0.113.66" a certes renvoye 29 documents, mais ceux-ci relevaient des categories web et intrusion_detection, pas authentication : ce n'etait donc pas la bonne IP pour repondre a la question posee.



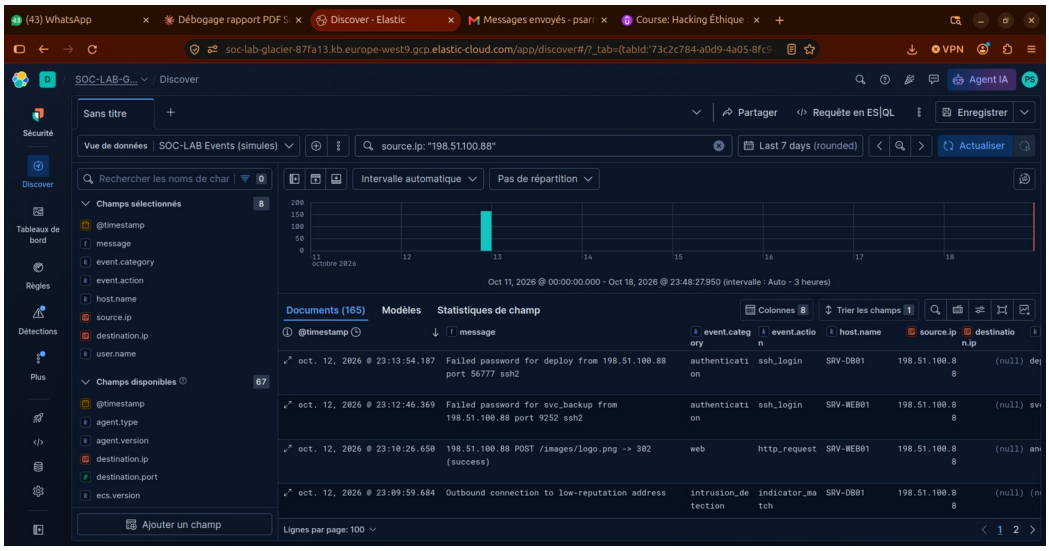
Premiere tentative (erronee) : source.ip: "203.0.113.66", 29 documents, mais de categorie web/intrusion_detection et non authentication -- IP reprise par erreur de la requete precedente plutot que lue dans le Top values du bon contexte.

Methode corrigee : avec le filtre event.category: "authentication" and event.outcome: "failure" actif (165 documents), un clic sur le champ source.ip dans le panneau de gauche de Discover ouvre son panneau Top values, qui agrege les valeurs les plus frequentes du champ sur le jeu de resultats filtre.

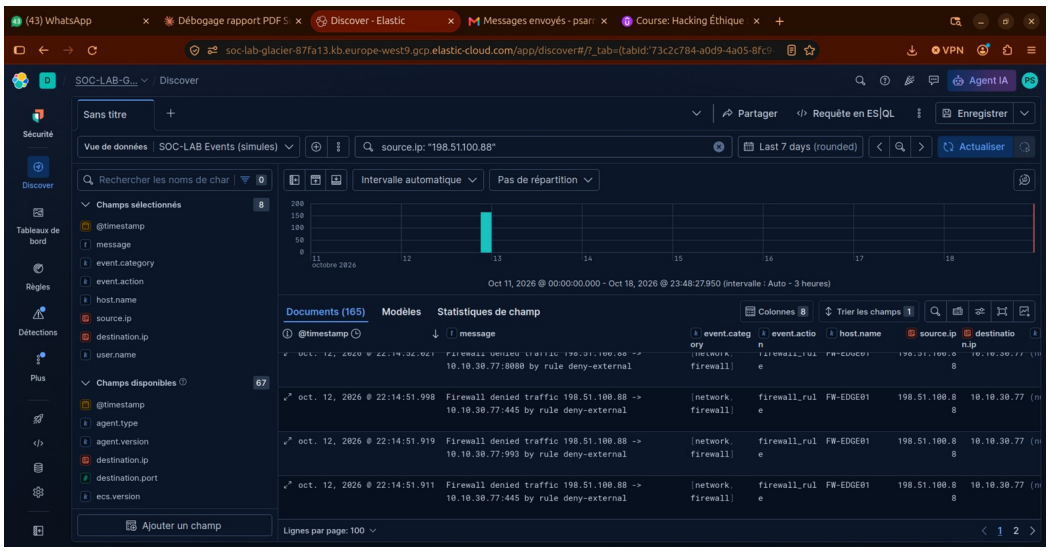


Top values de source.ip sur les 165 echecs d'authentification : 198.51.100.88 concentre 39,4% des evenements, tres largement devant les IP suivantes (10.10.10.11 a 6,7%).

L'adresse 198.51.100.88 concentre 39,4% des 165 echecs (environ 65 evenements), une concentration nette caracteristique d'un brute force automatise. La requete source.ip: "198.51.100.88" (sans filtre de categorie) montre que cette meme IP ne se limite pas a l'authentification : elle touche egalement les categories web et intrusion_detection, et genere du trafic bloque par le pare-feu FW-EDGE01.



source.ip: "198.51.100.88" sans filtre de categorie : echecs SSH sur SRV-DB01 et SRV-WEB01, requetes web, et une connexion sortante signalee vers une adresse a faible reputation.

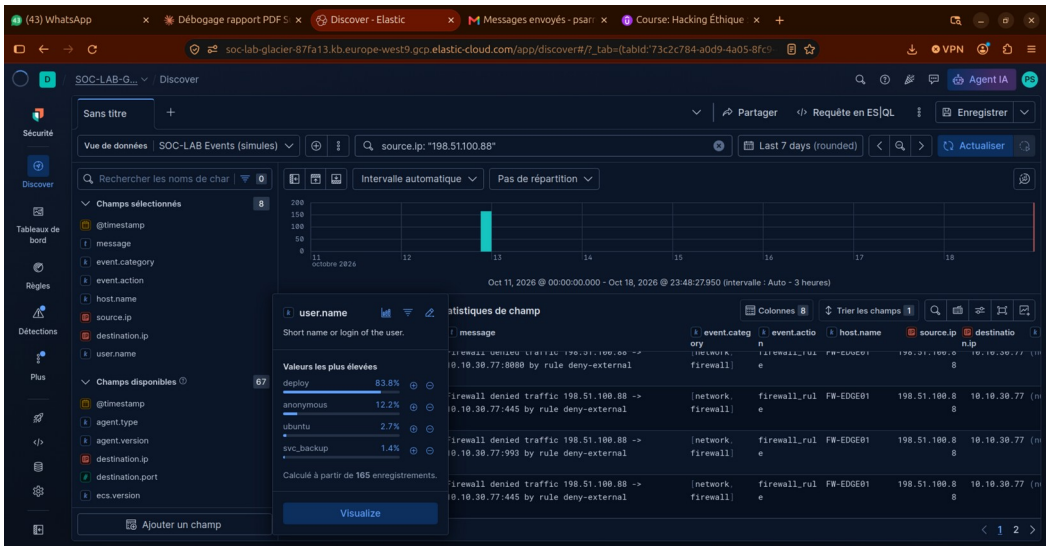


Evenements firewall_rule sur FW-EDGE01 : trafic 198.51.100.88 -> 10.10.30.77 bloque sur plusieurs ports (8080, 445, 993) par la regle deny-external, signature d'un balayage de ports complementaire au brute force.

IP concentrant le plus d'echecs, interne ou externe (Q8.2)

198.51.100.88 concentre 39,4% des 165 echecs d'authentification. Le champ source.geo.country_name s'est revele vide pour l'ensemble du dataset (absence d'enrichissement GeoIP dans les donnees simulees), la reponse s'appuie donc sur l'analyse de la plage d'adresse : 198.51.100.0/24 est une plage reservee par l'IANA a la documentation et aux tests (RFC 5737, TEST-NET-2), simulant ici une adresse externe au reseau, par opposition aux adresses en 10.0.0.0/8 (RFC 1918, privees/internes) observees ailleurs dans le dataset.

Hotes et comptes cibles par cette IP (Q8.3)



Top values de user.name sur source.ip: "198.51.100.88" : deploy a 83,8%, loin devant anonymous (12,2%), ubuntu (2,7%) et svc_backup (1,4%).

Hotes touches (Top values de host.name) : FW-EDGE01 (52,1%, trafic bloque/journalise), SRV-DB01 (39,4%, cible principale du brute force SSH), SRV-WEB01 (6,1%), SRV-FILE01 (1,2%), WKST-ALICE et kali-attacker (0,6% chacun). Comptes cibles (Top values de user.name) : deploy, tres largement majoritaire a 83,8%, suivi d'anonymous (12,2%), ubuntu (2,7%) et svc_backup (1,4%). Le compte deploy, compte de service utilise pour l'automatisation, est donc la cible privilegiee de cette campagne, un choix coherent avec un brute force automatise cherchant des identifiants faibles ou reutilises sur des comptes techniques plutot que sur des comptes personnels.

8.4 - Sequence EQL (optionnel)

Cette requete optionnelle, visant a detecter une sequence echec puis succes d'authentification pour une meme IP dans une fenetre de 15 minutes (POST /soc-lab-events-*/_eql/search avec une clause sequence by source.ip), n'a pas ete executee faute de temps disponible en fin de seance. Elle reste une piste d'approfondissement naturelle : son resultat determinerait si le brute force observe a fini par aboutir a une connexion reussie, ce qui ferait basculer l'incident d'une simple tentative a une compromission averee.

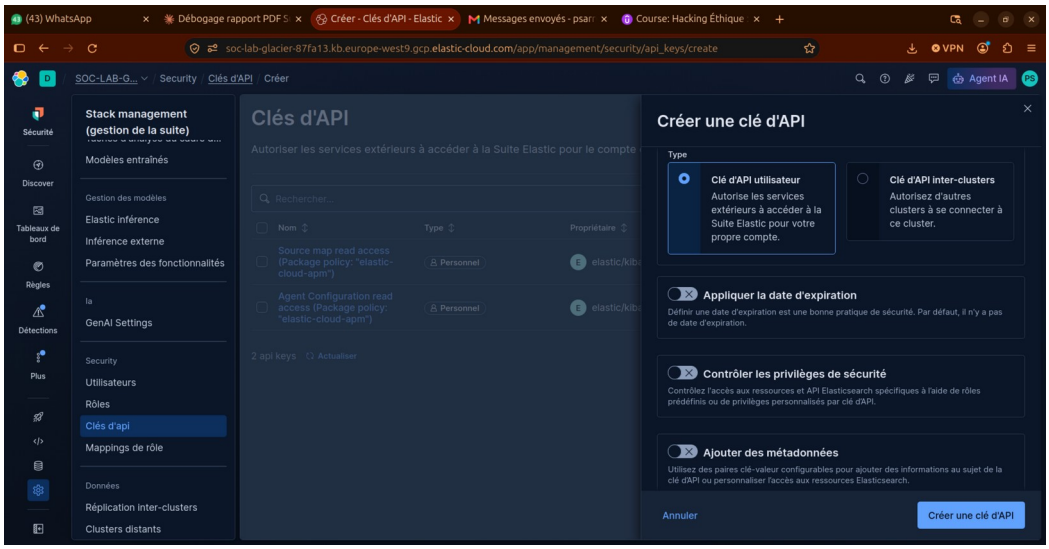
Point de controle 6

Des requetes KQL a deux conditions ont ete ecrites et executees avec succes, et une adresse IP source suspecte (198.51.100.88) a ete profilee de bout en bout : volume, hotes cibles, comptes vises, et nature probable (externe) de l'adresse.

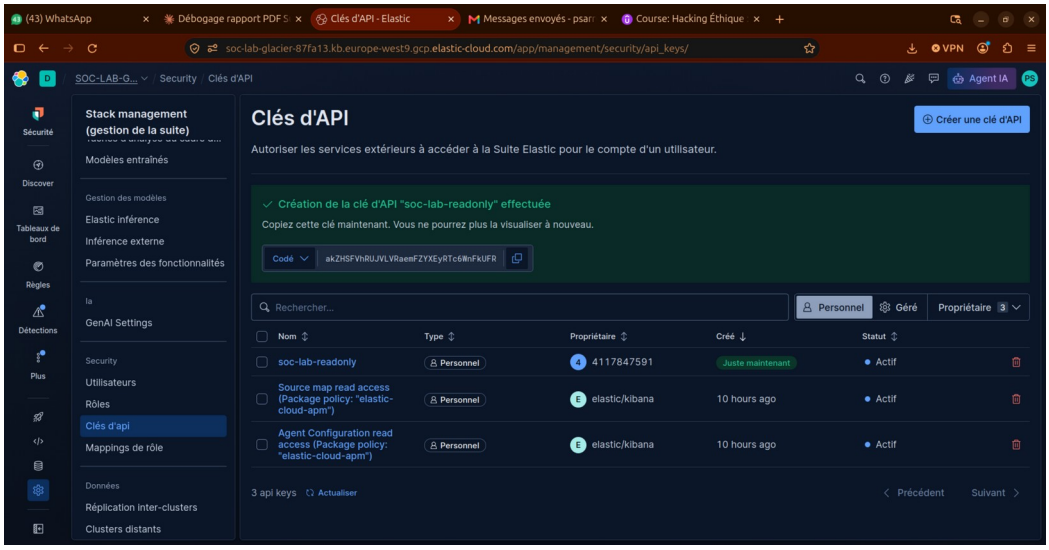
Partie VII : Cle API restreinte

La cle a ete creee depuis Stack Management > Security > Clefs d'API > Creer une cle d'API, avec le type Cle d'API utilisateur (par defaut, et correct pour ce cas d'usage), le nom soc-lab-readonly, et l'option Controler les privileges de securite activee pour y coller le descripteur de role suivant, restreignant la cle au strict necessaire :

```
{
  "soc-lab-readonly-role": {
    "cluster": [],
    "indices": [
      {
        "names": ["soc-lab-events-*"],
        "privileges": ["read", "view_index_metadata"]
      }
    ]
  }
}
```



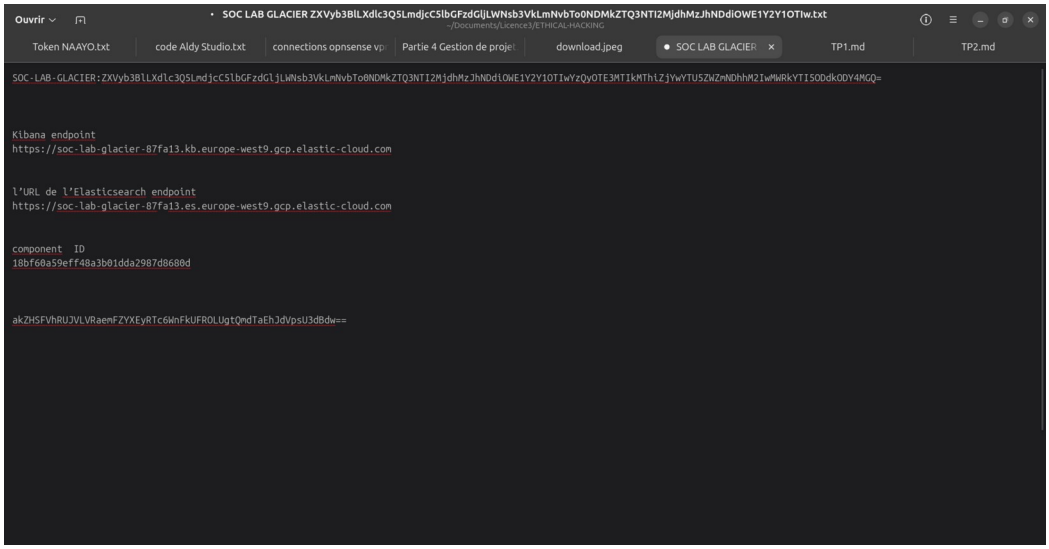
Formulaire de creation de cle API : type Cle d'API utilisateur, section Controler les privileges de securite disponible.



Confirmation de creation : "soc-lab-readonly" listee parmi les cles API, statut Actif, avec le message rappelant que la cle en clair ne sera plus jamais reaffichee.

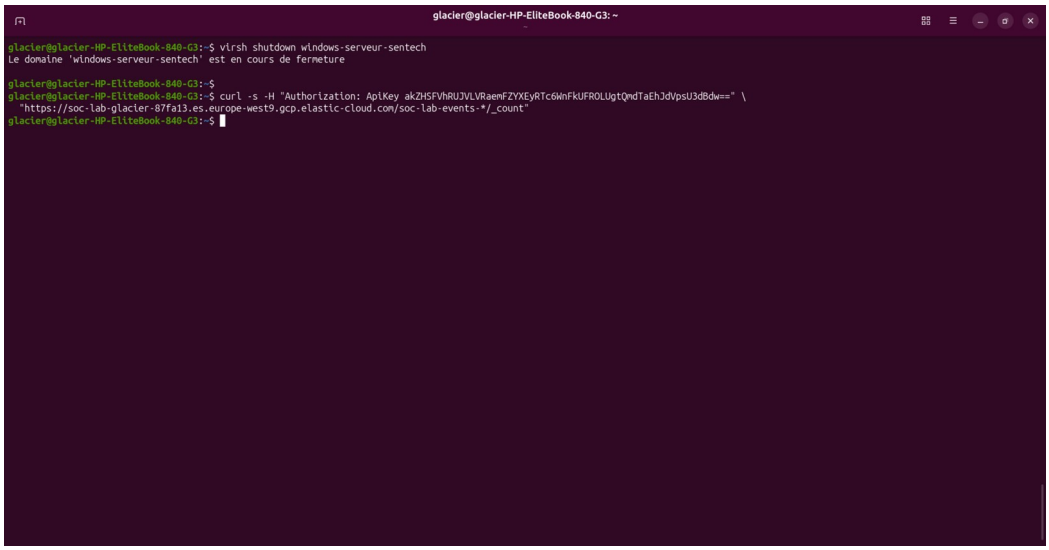
Test de la cle en lecture

Une premiere tentative d'acces, effectuee en ouvrant directement l'URL de l'endpoint Elasticsearch dans le navigateur plutot qu'en ligne de commande, a declenche la fenetre d'authentification HTTP Basic native du navigateur : attendu, puisque le navigateur reagit a un code 401 renvoye par Elasticsearch en proposant sa propre invite de connexion, independamment de toute cle API. Le test a ete repris correctement en terminal, avec curl et l'en-tete Authorization: ApiKey.



Fenetre d'authentification HTTP Basic du navigateur, declenchee par une ouverture directe de l'URL Elasticsearch -- confusion clarifiee, le test se fait en ligne de commande, pas dans le navigateur.

Un premier essai en curl avec l'option -s (mode silencieux) n'a affiche aucune sortie, ni resultat ni erreur -- cette option supprime en effet aussi bien la barre de progression que les messages d'erreur curl, rendant le diagnostic impossible en cas d'echec reseau transitoire.

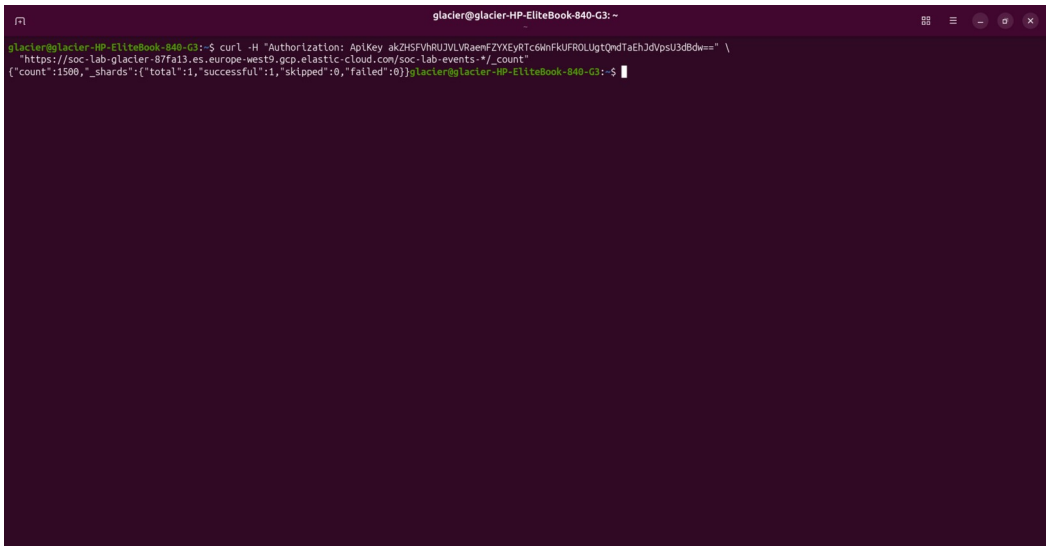


curl -s sans sortie visible : le mode silencieux masque aussi bien un succes vide qu'une erreur reseau.

La commande relancee sans -s a confirme le bon fonctionnement de la cle :

```
curl -H "Authorization: ApiKey <cle_encodee>" \
  "https://soc-lab-glacier-87fa13.es.europe-west9.gcp.elastic-cloud.com/soc-lab-events-
*/_count"

{"count":1500,"_shards":{"total":1,"successful":1,"skipped":0,"failed":0}}
```



Reponse en clair : {"count":1500,...}, confirmant que la cle en lecture seule retrouve bien les 1500 documents indexes.

Verification du moindre privilege

Pour verifier concretement que la restriction fonctionne, une tentative de suppression d'index a ete effectuee avec la meme cle :

```
curl -X DELETE -H "Authorization: ApiKey <cle_encodee>" \
  "https://soc-lab-glacier-87fa13.es.europe-west9.gcp.elastic-cloud.com/soc-lab-events-2026.10.18"
```

Elasticsearch a refuse l'action avec un code 403 et un message explicite : action [indices:admin/delete] is unauthorized for API key ... this action is granted by the index privileges [delete_index,manage,all]. La preuve est directe : la cle ne possede ni delete_index, ni manage, ni all, uniquement read et view_index_metadata, exactement comme configure.



curl -X DELETE refuse avec un code 403 security_exception : preuve concrete du moindre privilege applique a la cle soc-lab-readonly.

Pourquoi ne pas reutiliser le compte elastic (Q9.1)

Le compte elastic est un superutilisateur avec des droits d'administration totaux (creation/suppression d'index, gestion des utilisateurs, configuration du cluster). Un analyste SOC n'a besoin que de lire les donnees deja indexees. En cas de compromission des identifiants ou du poste de l'analyste, un acces elastic compromis met en peril l'ensemble de la plateforme, alors qu'une cle en lecture seule compromise ne permet,

dans le pire des cas, que de consulter des logs deja presents -- demontre concretement ci-dessus par l'echec de la tentative de suppression.

Cle API contre utilisateur (Q9.2)

Un utilisateur (comme elastic) est une identite humaine, avec nom et mot de passe, destinee a une session interactive dans Kibana. Une cle API est une credential machine-a-machine : sans session interactive, restreignable finement par privileges et par pattern d'index, assortie d'une date d'expiration optionnelle, et surtout revocable individuellement sans affecter les comptes utilisateurs. C'est le format adapte aux scripts et outils automatises, comme demontre par le test curl ci-dessus.

Revocation d'une cle compromise (Q9.3)

Depuis Stack Management > Security > Clefs d'API, un clic sur l'icone de suppression (poubelle) en bout de ligne revoque immediatement la cle correspondante : toute requete l'utilisant ensuite recoit un code 401 Unauthorized.

Point de controle 7

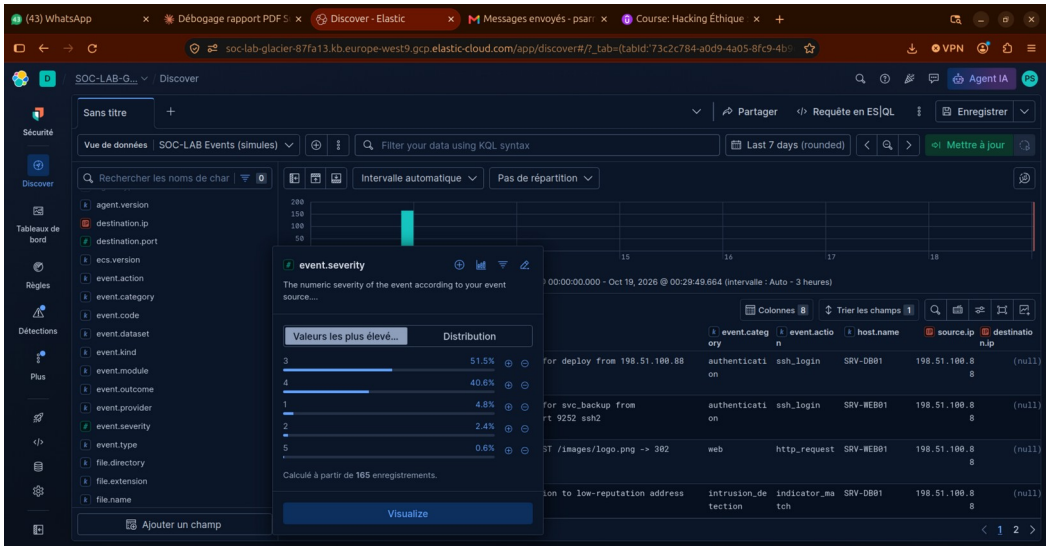
La cle soc-lab-readonly existe, repond correctement en lecture (count: 1500) et refuse explicitement toute action d'ecriture (403 sur une tentative de suppression), confirmant un acces strictement en lecture seule.

Partie VIII : Defis (autonomie)

Les quatre defis proposes par l'enonce ont ete realises, depassant le minimum de deux demande par la checklist de fin de TP.

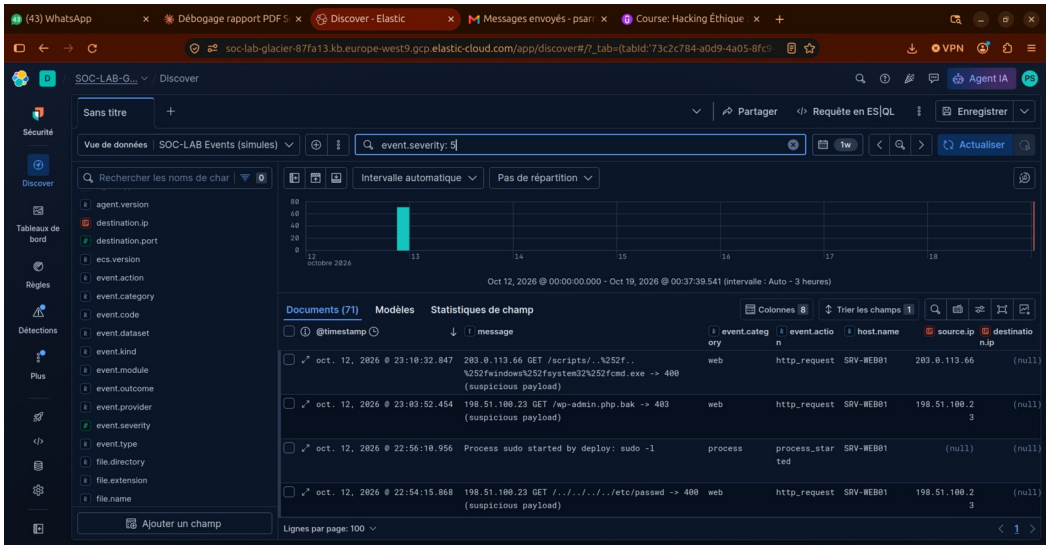
Defi A : le plus grave (event.severity maximal)

Le champ event.severity va de 1 a 5 dans ce dataset. Son panneau Top values indique une distribution de 51,5% (severite 3), 40,6% (severite 4), 4,8% (severite 1), 2,4% (severite 2) et seulement 0,6% pour la severite maximale (5) -- un sous-ensemble volontairement rare, cible via la requete event.severity: 5.

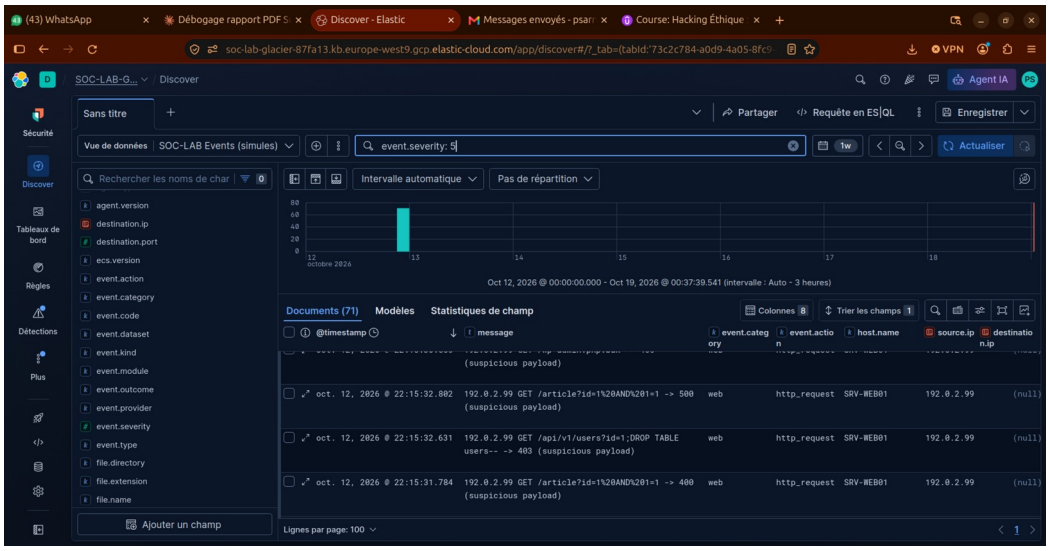


Distribution d'event.severity sur l'ensemble des documents : la severite 5 ne represente qu'une faible part des evenements, signe d'une classification selective.

71 documents correspondent a event.severity: 5. La grande majorite est constituee d'attaques web contre SRV-WEB01 (injection SQL, traversee de repertoire, sondage de fichiers sensibles) menees par trois adresses externes distinctes (203.0.113.66, 198.51.100.23, 192.0.2.99).



Premiers resultats event.severity: 5 : tentative d'execution de cmd.exe par traversee de repertoire, sondage de wp-admin.php.bak, et lecture de /etc/passwd, tous contre SRV-WEB01.

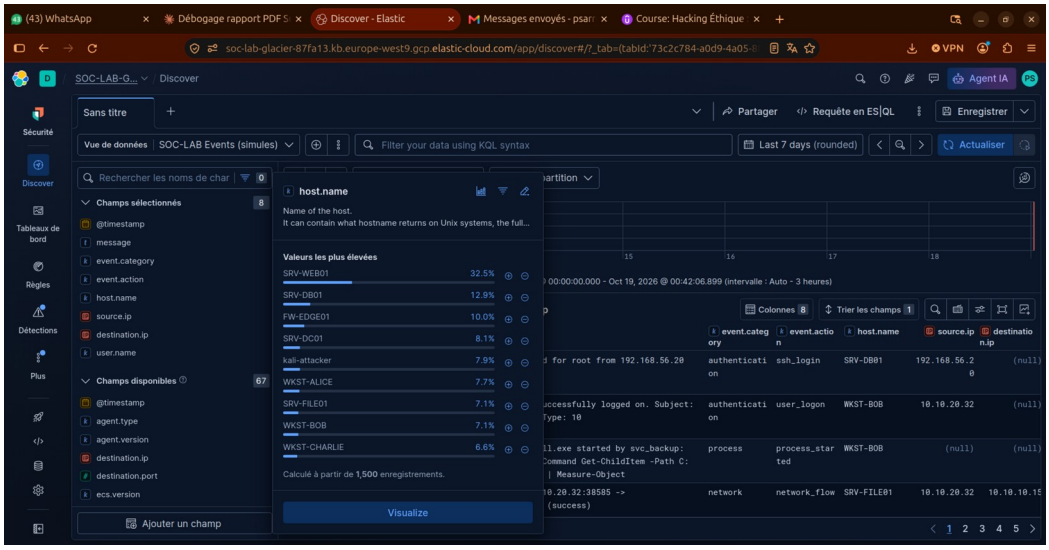


Suite des resultats : injections SQL explicites (DROP TABLE users--, id=1%20AND%201=1) depuis 192.0.2.99 contre SRV-WEB01.

Un seul evenement de severite 5 detonne dans cet ensemble : "Process sudo started by deploy: sudo -l" sur SRV-WEB01, classe event.category: process. Sa presence au meme niveau de severite que des attaques web caracterisees en fait un excellent cas d'etude pour le Defi C, traite plus loin.

Conclusion du Defi A : l'evenement de severite maximale la plus representative du dataset est une campagne d'injection SQL et de traversee de repertoire contre SRV-WEB01, menee depuis plusieurs adresses IP externes sur une fenetre concentree d'environ une heure, le 12 octobre en soiree.

Defi B : inventaire des hotes



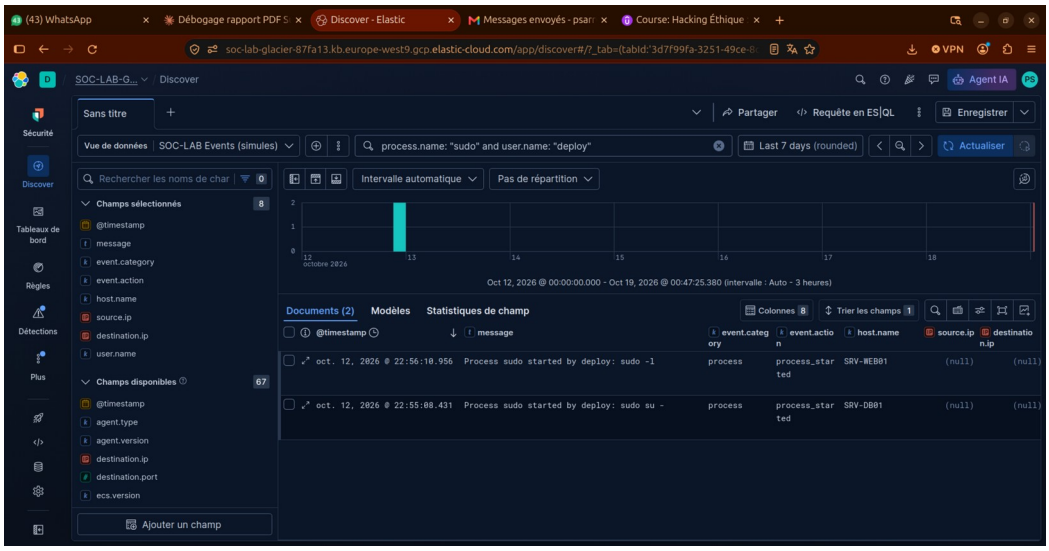
Top values de host.name calcule sur l'ensemble des 1500 documents : 9 valeurs distinctes totalisant environ 100% des evenements, sans categorie "Autre" residuelle.

9 valeurs distinctes de host.name ont ete denombrees sur les 1500 documents (7 jours) : SRV-WEB01 (32,5%), SRV-DB01 (12,9%), FW-EDGE01 (10,0%), SRV-DC01 (8,1%), kali-attacker (7,9%), WKST-ALICE (7,7%), SRV-FILE01 (7,1%), WKST-BOB (7,1%) et WKST-CHARLIE (6,6%). Ces pourcentages totalisant environ 100% sans categorie residuelle Autre, le decompte est exhaustif sans necessiter d'agregation cardinality supplementaire via Dev Tools. Le parc simule reconstitue ainsi quatre serveurs (web, base de donnees, fichiers, controleur de domaine), un pare-feu perimetrique, trois

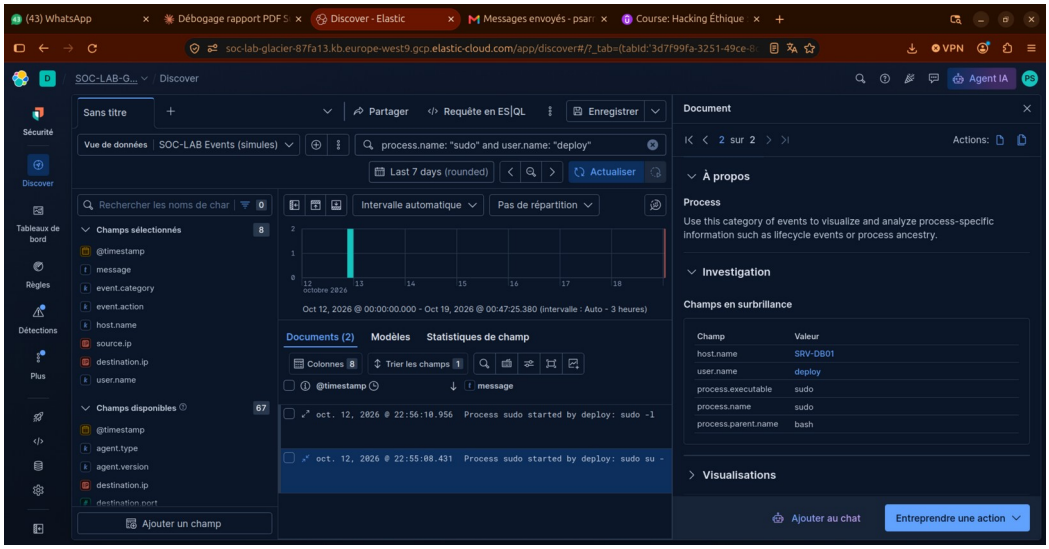
postes utilisateurs, et une machine nommee explicitement kali-attacker, correspondant a la machine de l'attaquant dans le scenario pedagogique.

Defi C : faux positif

Deux evenements process ont ete isoles avec la requete process.name: "sudo" and user.name: "deploy" : sudo -l sur SRV-WEB01 (22:56:10) et sudo su - sur SRV-DB01 (22:55:08), tous deux classes severite 5.



Les deux evenements sudo du compte deploy : sudo -l sur SRV-WEB01 et sudo su - (plus sensible) sur SRV-DB01.



Document developpe de l'evenement sudo su - sur SRV-DB01 : process.parent.name vaut bash, signature d'une session shell normale plutot que d'un processus d'exploitation.

A premiere vue, ces evenements ressemblent a une elevation de privileges suspecte : compte de service deploy (deja identifie comme cible du brute force SSH en Partie VI), commande sensible (sudo su - ouvre un shell root complet), severite maximale. Plusieurs elements renversent pourtant cette premiere lecture. D'abord la chronologie : ces deux sudo ont lieu a 22h55-22h56, soit avant le debut des tentatives de brute force observees sur ce meme compte (23h09-23h14) -- un sudo consecutif a une compromission reussie serait au contraire attendu apres un succes d'authentification, pas quinze a vingt minutes avant la premiere tentative d'echec. Ensuite le detail technique : process.parent.name vaut bash, signature d'une session shell interactive normale, et non d'un processus d'exploitation ou d'un payload injecte. Enfin le

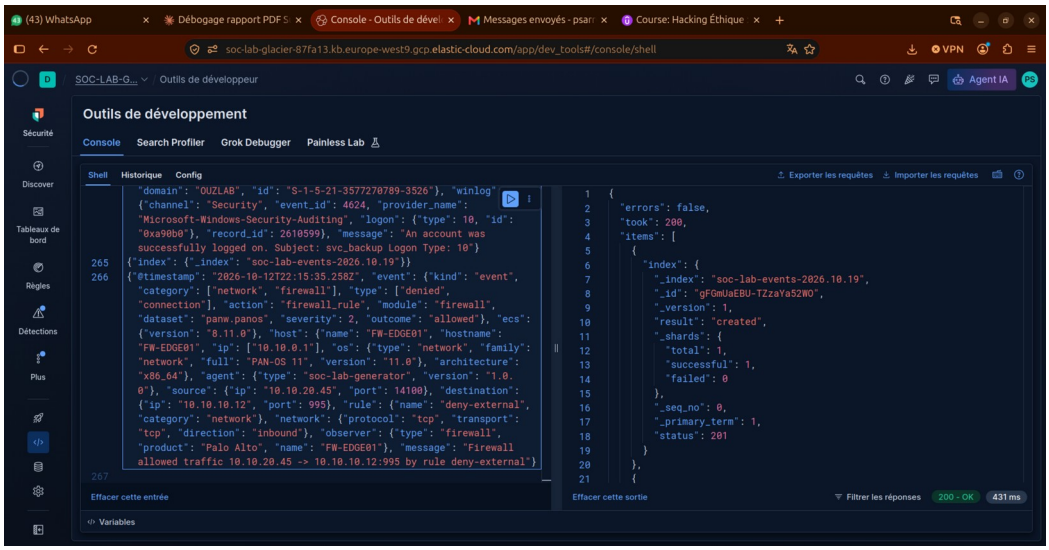
contexte : ce schema est coherent avec un script de deploiement automatise utilisant ce compte de service pour des taches d'administration routinieres.

Conclusion du Defi C : l'evenement sudo su - du compte deploy sur SRV-DB01 est un faux positif plausible, une activite d'administration legitime survenant avant l'attaque et portee par un compte dont le nom, facilement devinable, en fait neanmoins une cible privilegiee pour un attaquant -- un point a signaler dans les recommandations de durcissement (renommage ou rotation des identifiants des comptes de service standards).

Defi D : ingestion scriptee via _bulk

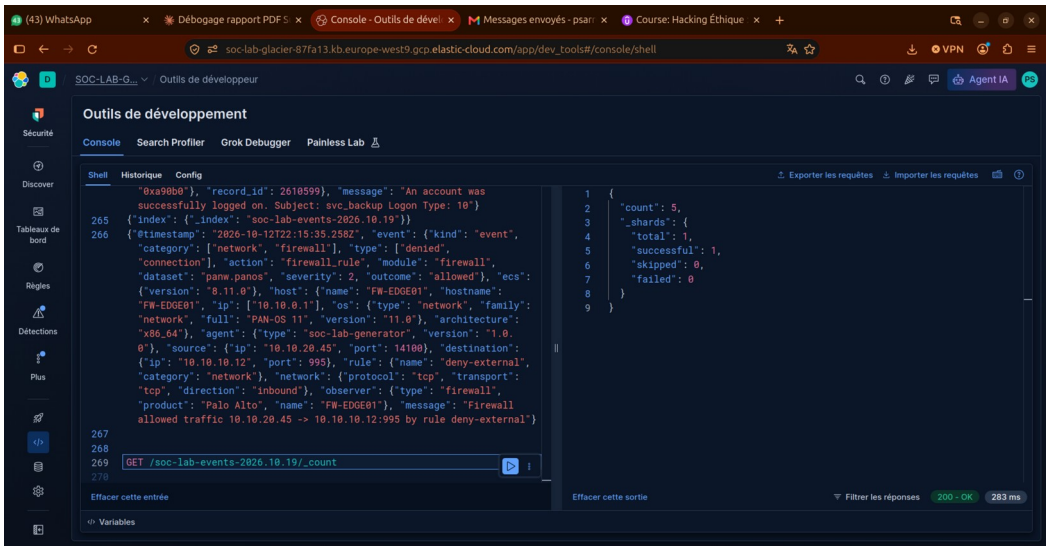
Cinq evenements reels, extraits des premieres lignes de sample-normal.ndjson, ont ete convertis au format _bulk de l'API Elasticsearch. Ce format exige une ligne d'action-metadonnee avant chaque document (par exemple {"index": {"_index": "..."}}), contrairement au format NDJSON source ou chaque ligne est directement un document. La cible choisie, soc-lab-events-2026.10.19 (date du jour de l'exercice), respecte le pattern soc-lab-events-* du template applique en Partie II, qui s'y applique donc automatiquement -- une demonstration concrete, en conditions reelles, de la reponse a la question Q4.3.

```
POST /_bulk
{"index": {"_index": "soc-lab-events-2026.10.19"}}
{"@timestamp": "2026-10-12T22:14:35.328Z", "event": {...}, "host": {"name": "WKST-
BOB", ...}, ...}
{"index": {"_index": "soc-lab-events-2026.10.19"}}
{"@timestamp": "2026-10-12T22:14:56.785Z", "event": {...}, "host": {"name": "SRV-
WEB01", ...}, ...}
... (5 paires action/document au total)
```



Reponse de POST /_bulk : "errors": false, 5 elements avec "result": "created" et "status": 201 chacun.

La reponse confirme errors: false et un result: created pour chacun des 5 documents (status 201). Une verification finale via GET /soc-lab-events-2026.10.19/_count renvoie count: 5, confirmant que l'ingestion scriptee a parfaitement reussi, sans dupliquer les 1500 documents deja analyses dans l'index principal.



GET /soc-lab-events-2026.10.19/_count : {"count": 5}, confirmant l'ingestion reussie des 5 documents via l'API _bulk.

Point de controle defis

Les quatre defis ont ete menes a terme : identification de la campagne d'attaque web de severite maximale (Defi A), inventaire exhaustif des 9 hotes du parc simule (Defi B), qualification argumentee d'un faux positif d'administration (Defi C), et demonstration reussie d'une ingestion scriptee via l'API _bulk (Defi D).

Conclusion

Ce TP2 a permis d'alimenter de bout en bout la plateforme SOC-LAB-GLACIER, jusque-la vide : application d'un template d'index ECS garantissant des mappings corrects, upload des cinq datasets simules via Kibana, verification systematique de l'ingestion dans Dev Tools, creation d'une Data View a identifiant personnalise (condition necessaire aux dashboards du TP4), premieres recherches KQL permettant de profiler une campagne de brute force SSH jusqu'a son compte et ses hotes cibles, et creation d'une cle API en lecture seule appliquant concretement le principe de moindre privilege.

Plusieurs obstacles reels ont jalonne ce TP et ont ete documentes tels qu'ils se sont presentes plutot que masques : une confusion entre les champs Index name et Data view name au premier upload, un bug de double barre oblique dans une URL tapee a la main provoquant une erreur 404, une reutilisation erronee d'une adresse IP lue dans le mauvais contexte lors du suivi d'une IP source, une commande curl en mode silencieux masquant temporairement un resultat, et une Data View incorrecte selectionnee par inadvertance au moment d'examiner l'evenement sudo du Defi C. Chacune de ces erreurs a ete identifiee, expliquee et corrigee en suivant une methode reproductible, ce qui constitue en soi une competence d'analyste SOC aussi importante que la maitrise technique des outils.

Les quatre defis d'autonomie ont permis d'aller au-dela du strict minimum demande : une campagne d'attaque web caracterisee (Defi A), un inventaire exhaustif du parc (Defi B), une premiere qualification argumentee d'un faux positif (Defi C), et la demonstration d'une methode d'ingestion automatisable par script (Defi D), en reponse directe a la limite de l'upload manuel identifiee en Partie III (Q5.3).

La plateforme contient desormais 1505 documents repartis sur deux index (1500 dans soc-lab-events-2026.10.18, 5 dans soc-lab-events-2026.10.19), une Data View correctement configuree, et une premiere piste d'activite suspecte solidement etayee (l'adresse 198.51.100.88 et le compte deploy). C'est sur cette base que s'appuiera le TP3, consacre a la collecte de donnees reelles via un agent Elastic connecte au Fleet Server heberge.